

Decomposition: Plans That Grow When the World Talks Back

Chapter 5, objective 1.5. Static versus dynamic decomposition. Exhibits E5 and E8 decoded, and the growth cap that separates a planner from a spiral.

Packing versus walking. You can pack for a Himalayan trek in advance. The list is knowable. You cannot walk it in advance: at the river crossing you learn the bridge is out and the plan grows a detour. Static decomposition is packing: fixed subtasks, E2's assembly line. Dynamic decomposition is walking: subtasks appear because a result revealed them. Your 75 percent row was exactly this distinction, a plan that adapts versus a sequence that executes regardless.

Exhibit E5: the foreman writes the plan

Lines	What E5 does	In plain words
1 to 5	orchestrate(brief): plan = ask_json(main model, brief plus list 2 to 5 subtasks as JSON with id, task, needs).	KF-2481's customer is threatening to leave, fix it. The brief goes to the foreman, whose first paid act is writing the task list. Compare E2, where VALID and the function order were typed by Arjun. Here the subtasks are model-made and different for every brief. This single line is the boundary between 1.3's workflows and 1.5's decomposition.
6 to 10	findings = {}. For each subtask, gather the findings it needs, then ask the small model to do the subtask with those known facts.	The site book. Each subtask runs with its declared dependencies injected and nothing else. A worker checking can we re-route reads the status finding first. Workers are small models: the foreman thinks, the crew executes, the payroll stays sane.
11 to 13	Return ask(main model, brief plus all findings plus synthesise one recommendation for Meera).	The foreman reads every chit and writes one recommendation. Not a list of findings. A decision. Independent subtasks here could be E4's gather; patterns compose.

Exhibit E8: re-planning, the bridge is out

E5's plan, once written, still executes fixedly. E8 adds the trek's truth: a finding can change the plan. This is dynamic decomposition complete, and the book says to type it slowest.

Lines	What E8 does	In plain words
1 to 3	replan(brief, max_new=3). tasks = plan_from(brief). findings = {}, grew = 0.	The trek begins with E5's packed list, and a second budget: max_new = 3. A plan that can grow needs a cap on growth, or re-planning becomes the unbounded loop wearing a planner's clothing. Law 3 applies to plans, not just turns.
4 to 7	while tasks: pop the front task, run the worker with findings so far, file the result.	Walk one step. Take the front task, run it, file the finding. So far this is E5 with a queue.
8 to 12	verdict = ask_json(main model, given this new finding, does the remaining plan still hold? Reply ok true or false, add new subtasks, drop ids), sent with only the finding and the remaining tasks.	The walk-and-look. After every finding, one focused question: does the rest of the plan survive this? The model sees the new finding and the remaining tasks, not the whole history. Cheap, specific, structured. This call is what subtasks that adapt as new information is discovered means in running code.
13 to 17	if not ok and grew under max_new: drop the listed tasks, add the new ones, grew += number added.	The bridge is out. Customs hold discovered, so the re-route quote (pointless until release) is dropped, and file e-way bill and notify Padma appear. grew counts the detours against the cap. The plan changed because a result said so. Your 75 percent row, closed.
18	Return synthesise(brief, findings).	Same ending as E5. Decomposition patterns end the way they must: one recommendation, traceable to findings.

The expected trace, in the canned world

```
# run 1: status check -> CUSTOMS_HOLD, missing e-way bill
# replan: ok=false drop=[3] add=["file e-way bill", "notify Padma of ETA"]
# run 2: file e-way bill -> filed, release ~24h
# run 3: notify Padma -> draft ready
# grew=2, budget max_new=3 not exhausted
RECOMMENDATION: file the e-way bill today; hold the re-route (costlier
than the 24h release); send Padma the revised ETA with a goodwill note.
```

Read it against the Atlas A6 tree: subtasks 1 and 2 were planned (solid boxes), subtask 3 was planned and died the moment the hold was found (crossed out), subtasks 4 and 5 appeared because a finding said so (dashed boxes). If that tree could have been drawn before the run, chaining would have sufficed and orchestrator-workers was over-engineering. The dashed boxes are the exam's evidence that dynamic machinery earned its keep.

Sabotage lab

Change the growth check to always true and seed the replanner to always return ok false with one new subtask. Count the turns before you kill it. Lesson: **the growth cap is the difference between a planner and a spiral.** Fix it back.

LAW 4 · Decompose on discovery, not in advance. When subtasks are knowable, decompose in code (chaining), cheapest and most testable. When they aren't, let the model write the plan (orchestrator-workers). When findings can invalidate the plan, re-plan on findings with a growth cap. Escalating uncertainty, escalating machinery, never more than the stem demands.

Uncertainty level	Machinery	Exhibit	Budget organ
Subtasks knowable before the run	chain in code	E2	the fixed list is the budget
Subtasks unknowable, plan writable once	orchestrator-workers	E5	2 to 5 subtasks, declared needs
Findings can invalidate the plan mid-run	re-planning	E8	max_new growth cap

Trap: decomposing the known. A stem whose subtasks are listed in the stem itself does not need orchestrator-workers. The decomposition already exists; paying a model to rediscover it is over-engineering with extra steps. Dynamic machinery is for uncertainty that survives reading the brief twice.

Domain 3 connection: the worker that runs status check is calling the lab's E1 tool through the E3 gate. The worker that files the e-way bill would read SOP 214 section 4 through the E4 and E5 retrieval. Decomposition decides which workers exist. Domain 3 decides how each worker reaches its facts.

***The one thing to remember.** Three levels of uncertainty, three levels of machinery. Chain what you can list. Let the model plan what you cannot. Re-plan on findings only when findings can really change the plan, and cap the growth. The dashed boxes in the tree are your proof.*